

BALDIRIS

MANUAL MAESTRO DE CONSTRUCCIÓN DEL PROYECTO

Plataforma tecnológica empresarial / SaaS

Versión: 1.0

Estado: Desarrollo inicial

Objetivo: Construcción de una empresa tecnológica y de una plataforma central para administrar múltiples negocios y servicios tecnológicos.

DOCUMENTO BASE PARA CODEX

Este documento debe utilizarse como especificación principal del proyecto. Debe leerse antes de comenzar a modificar o crear código.

CONTENIDO

1. Instrucción principal para Codex
2. ¿Qué es BALDIRIS?
3. Visión del proyecto
4. Objetivo principal
5. Concepto multi-tenant
6. Primer producto de BALDIRIS
7. K.E. Chicharrón
8. Administración de BALDIRIS
9. Dashboard principal de BALDIRIS
10. Ingresos
11. Gastos
12. Reservas
13. Utilidades
14. Colaboradores
15. Pagos
16. Clientes
17. Proyectos
18. Negocios
19. Usuarios y roles
20. Seguridad
21. Base de datos
22. Principio fundamental de la base de datos
23. BALDIRIS POS
24. Experiencia del usuario
25. Identidad visual

26. Estructura del proyecto
27. AGENTS.MD
28. Desarrollo por fases
29. Infraestructura
30. Dominio
31. Producción
32. GitHub
33. Backups
34. Escalabilidad
35. Futuros productos
36. Modelo comercial
37. Propiedad del software
38. Regla para nuevos clientes
39. Ejemplo de crecimiento
40. Reglas para Codex
41. Metodología de trabajo
42. Calidad
43. Estado inicial
44. Primer objetivo técnico
45. Primer objetivo comercial
46. Principio de crecimiento
47. Sitio corporativo futuro
48. Separación entre sitio web y plataforma
49. Futura aplicación móvil
50. Resultado final esperado
51. Orden de implementación obligatorio

52. Instrucción final para Codex

53. Frase central del proyecto

1. INSTRUCCIÓN PRINCIPAL PARA CODEX

Este documento contiene las especificaciones generales del proyecto BALDIRIS.

Antes de escribir código, debes leer y comprender completamente este documento.

Tu función será actuar como desarrollador principal del proyecto BALDIRIS, trabajando directamente sobre el proyecto abierto en Visual Studio Code.

No debes limitarte a generar ejemplos de código. Debes:

- Analizar la estructura existente del proyecto.
- Crear los archivos necesarios.
- Modificar los archivos existentes cuando corresponda.
- Ejecutar pruebas.
- Detectar y corregir errores.
- Mantener una arquitectura organizada.
- Evitar duplicación innecesaria.
- Mantener el código preparado para crecimiento.
- Documentar decisiones técnicas importantes.
- No eliminar funcionalidades existentes sin autorización.
- No reemplazar una funcionalidad funcional por otra sin comprobar primero que la nueva funciona correctamente.

IMPORTANTE: BALDIRIS no debe construirse como una página web simple. Debe construirse como una plataforma empresarial escalable, preparada para manejar diferentes negocios/clientes dentro de una misma infraestructura.

2. ¿QUÉ ES BALDIRIS?

BALDIRIS será una empresa tecnológica dedicada al desarrollo de:

- Software empresarial.
- Sistemas POS.
- Sistemas para restaurantes.
- Sistemas administrativos.
- Aplicaciones web.
- Aplicaciones móviles.
- Automatización.
- Inteligencia artificial.
- Sistemas para hoteles.
- Sistemas financieros y administrativos.
- Soluciones personalizadas para empresas.

BALDIRIS será la marca tecnológica principal. Los productos y sistemas desarrollados por BALDIRIS podrán utilizarse por diferentes empresas y negocios.

3. VISIÓN DEL PROYECTO

La visión es construir una plataforma tecnológica capaz de crecer junto con BALDIRIS.

Estructura conceptual:

```

BALDIRIS
|
+- Administración
|
+- Clientes
|
+- Proyectos
|
+- Finanzas
|
+- Colaboradores
|
+- Reportes
|
+-- Negocios
    |
    +- K.E. Chicharrón
    +- Restaurante 2
    +- Restaurante 3
    +- Futuras empresas
  
```

4. OBJETIVO PRINCIPAL

Crear una plataforma desde la cual el administrador de BALDIRIS pueda controlar la empresa y, al mismo tiempo, administrar los diferentes negocios que utilizan las soluciones desarrolladas por BALDIRIS.

Ejemplo: BALDIRIS -> Administración de BALDIRIS y Negocios -> K.E. Chicharrón, Restaurante 2, Restaurante 3. El administrador podrá entrar a un negocio y trabajar únicamente con la información correspondiente a ese negocio.

5. CONCEPTO MULTI-TENANT

La plataforma debe diseñarse siguiendo un concepto multi-tenant. Esto significa que diferentes empresas pueden utilizar la misma plataforma, pero sus datos deben permanecer completamente separados.

Los datos de K.E. Chicharrón nunca deben mezclarse con los datos de otro restaurante.

```

BALDIRIS
|
  
```

```

+- K.E. Chicharrón
|   +- Productos
|   +- Pedidos
|   +- Ventas
|   +- Reportes
|
+- Restaurante 2
|   +- Productos
|   +- Pedidos
|   +- Ventas
|   +- Reportes
|
+- Restaurante 3
   +- Productos
   +- Pedidos
   +- Ventas
   +- Reportes

```

6. PRIMER PRODUCTO DE BALDIRIS

El primer producto comercial será el sistema para restaurantes.

Nombre provisional: BALDIRIS POS. También puede utilizarse BALDIRIS Restaurant. Este producto será construido tomando como base el sistema actualmente desarrollado para K.E. Chicharrón. No se debe destruir el sistema existente; debe analizarse y reutilizarse la mayor cantidad posible de código funcional.

Producto inicial

BALDIRIS POS / BALDIRIS Restaurant será el producto de entrada al mercado y su primer caso de uso será K.E. Chicharrón.

7. K.E. CHICHARRÓN

K.E. Chicharrón será el primer negocio real integrado a BALDIRIS.

Su estructura será: Dashboard, Menú, Productos, Pedidos, Cocina, Ventas, Clientes y Reportes. El objetivo es que posteriormente sea posible agregar otro restaurante sin tener que desarrollar nuevamente todo el sistema.

8. ADMINISTRACIÓN DE BALDIRIS

El sistema administrativo interno de BALDIRIS será independiente de los sistemas de los clientes. Debe permitir administrar la empresa.

- Dashboard
- Ingresos
- Gastos

- Reservas
- Utilidades
- Colaboradores
- Pagos
- Proyectos
- Clientes
- Reportes
- Configuración

```

Administración
|
+- Dashboard
+- Ingresos
+- Gastos
+- Reservas
+- Utilidades
+- Colaboradores
+- Pagos
+- Proyectos
+- Clientes
+- Reportes
+- Configuración

```

9. DASHBOARD PRINCIPAL DE BALDIRIS

El Dashboard debe mostrar una visión general de la empresa.

- Ingresos del mes.
- Gastos del mes.
- Utilidad.
- Dinero reservado.
- Dinero disponible.
- Pagos pendientes.
- Clientes activos.
- Proyectos activos.
- Negocios activos.
- Colaboradores.
- Facturación por proyecto.
- Rentabilidad.

Ejemplo visual: Ingresos \$8.500.000; Gastos \$2.100.000; Reservas \$1.200.000; Utilidad \$5.200.000; Clientes activos 3; Proyectos activos 4; Negocios 3. El Dashboard debe utilizar tarjetas, gráficos y tablas cuando sea útil. Debe tener diseño profesional.

10. INGRESOS

El módulo de ingresos permitirá registrar y controlar todo el dinero recibido por BALDIRIS.

- Cliente.
- Proyecto.
- Negocio.
- Fecha.
- Método de pago.
- Concepto.
- Valor.
- Estado.
- Observaciones.

Ejemplo: Cliente K.E. Chicharrón; Proyecto Sistema POS; Concepto Implementación; Valor \$350.000; Fecha 15/09/2026; Estado Pagado.

11. GASTOS

Registrar todos los gastos de BALDIRIS.

- Software.
- Servidores.
- Dominios.
- Publicidad.
- Transporte.
- Equipos.
- Servicios.
- Pagos a colaboradores.
- Herramientas.
- Otros.

Cada gasto deberá incluir fecha, categoría, valor, descripción, proyecto relacionado, responsable y método de pago.

12. RESERVAS

BALDIRIS debe permitir separar dinero para reservas de la empresa.

Ingresos - Gastos = Utilidad. Utilidad - Reserva = Utilidad distribuible. La reserva podrá configurarse mediante un porcentaje. Ejemplo: Utilidad \$5.000.000; Reserva 30%; Reserva \$1.500.000; Utilidad distribuible \$3.500.000. El porcentaje debe ser configurable.

13. UTILIDADES

El sistema debe calcular automáticamente:

- Ingresos.
- Gastos.
- Utilidad bruta.
- Reservas.
- Utilidad disponible.
- Utilidad por proyecto.
- Utilidad por cliente.
- Utilidad por negocio.

14. COLABORADORES

BALDIRIS tendrá un módulo para administrar colaboradores.

- Nombre.
- Cargo.
- Rol.
- Proyecto.
- Forma de pago.
- Porcentaje si aplica.
- Pagos realizados.
- Pagos pendientes.
- Estado.

Ejemplo: Colaborador 1; Rol Desarrollo; Proyecto K.E. Chicharrón; Pago \$300.000; Estado Pendiente.

15. PAGOS

Debe existir un módulo para controlar:

- Pagos recibidos.
- Pagos realizados.
- Pagos pendientes.
- Pagos a colaboradores.
- Pagos de clientes.
- Pagos relacionados con proyectos.

16. CLIENTES

Cada cliente de BALDIRIS tendrá un registro.

- Nombre.
- Empresa.
- Teléfono.
- Correo.
- Dirección.
- Tipo de cliente.
- Estado.
- Proyectos.
- Pagos.
- Historial.

Ejemplo: Cliente K.E. Chicharrón; Proyecto Sistema POS; Estado Activo; Ingresos generados \$350.000; Servicio mensual \$79.900.

17. PROYECTOS

Cada proyecto deberá poder registrarse y administrarse.

- Nombre.
- Cliente.
- Fecha de inicio.
- Fecha de finalización.
- Valor.
- Estado.
- Gastos.
- Ingresos.
- Utilidad.
- Colaboradores.
- Observaciones.

Estados: Planificado, En desarrollo, En pruebas, Activo, Finalizado, Suspendido.

18. NEGOCIOS

Este módulo será uno de los más importantes. Permitirá registrar empresas/negocios que utilizan productos BALDIRIS.

Cada negocio tendrá identificador único, nombre, logo, información comercial, configuración, usuarios, productos, pedidos, ventas y reportes.

```
Negocios
|
+- K.E. Chicharrón
|   +- Menú
|   +- Pedidos
|   +- Cocina
```

```

|   +- Ventas
|   +- Reportes
|
+- Restaurante 2
|   +- Menú
|   +- Pedidos
|   +- Cocina
|   +- Ventas
|   +- Reportes
|
+- Restaurante 3
   +- Menú
   +- Pedidos
   +- Cocina
   +- Ventas
   +- Reportes

```

19. USUARIOS Y ROLES

Debe existir un sistema de autenticación.

Roles iniciales: SUPER ADMIN - control total de BALDIRIS; ADMINISTRADOR DE NEGOCIO - administra únicamente su negocio; COCINA - gestiona pedidos de cocina; EMPLEADO - permisos limitados; CLIENTE - acceso limitado si se implementa.

```

SUPER ADMIN
ADMINISTRADOR DE NEGOCIO
COCINA
EMPLEADO
CLIENTE

```

20. SEGURIDAD

La seguridad es obligatoria.

- Autenticación segura.
- Contraseñas cifradas/hasheadas.
- Control de sesiones.
- Autorización por roles.
- Protección de rutas.
- Validación de datos.
- Protección contra acceso entre negocios.
- Variables de entorno para secretos.
- No almacenar contraseñas en texto plano.
- No colocar claves privadas directamente en el código.
- No publicar credenciales en GitHub.

21. BASE DE DATOS

La arquitectura debe prepararse para una base de datos real.

Entidades principales conceptuales: users, roles, permissions, businesses, business_users, clients, projects, products, categories, orders, order_items, sales, expenses, income, reserves, collaborators, payments, reports, settings.

```

users
roles
permissions
businesses
business_users
clients
projects
products
categories
orders
order_items
sales
expenses
income
reserves
collaborators
payments
reports
settings

```

22. PRINCIPIO FUNDAMENTAL DE LA BASE DE DATOS

No crear una base de datos independiente manualmente para cada restaurante si la arquitectura elegida permite una solución multi-tenant segura. La plataforma debe diseñarse para crecer.

Ejemplo: businesses con ID 1 K.E. Chicharrón, ID 2 Restaurante 2, ID 3 Restaurante 3. Los productos, pedidos y demás registros estarán relacionados con el negocio correspondiente.

23. BALDIRIS POS

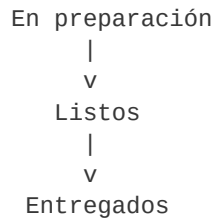
El sistema POS debe incluir inicialmente:

Menú: categorías, productos, fotografías, descripción, precio, disponibilidad, favoritos. Carrito: agregar, eliminar, modificar cantidades, total y confirmación. Pedidos: crear pedido, estado e historial. Cocina: Pedidos entrantes -> Por preparar -> En preparación -> Listos -> Entregados. Reportes: ventas, pedidos, productos, fechas y totales.

```

Pedidos entrantes
|
v
Por preparar
|
v

```



24. EXPERIENCIA DEL USUARIO

El diseño debe ser moderno, profesional, limpio, responsive, fácil de utilizar y compatible con celulares, computadores y tablets.

No utilizar interfaces excesivamente cargadas. BALDIRIS debe parecer un producto profesional y comercial.

25. IDENTIDAD VISUAL

Marca: BALDIRIS. El nombre debe escribirse siempre exactamente: BALDIRIS.

No utilizar variantes como Valdiris, Baldiri, BaldiriS, Valdiri o Baldyris. La identidad visual deberá poder evolucionar posteriormente. El diseño debe transmitir tecnología, confianza, innovación, profesionalismo y escalabilidad.

26. ESTRUCTURA DEL PROYECTO

La estructura final dependerá de la tecnología utilizada, pero debe mantener separación clara entre frontend, backend, base de datos, autenticación, servicios, componentes, módulos y configuración.

Estructura conceptual: BALDIRIS/ -> README.md, AGENTS.md, .env.example, .gitignore, docs/ (arquitectura.md, base-datos.md, seguridad.md, roadmap.md), frontend/, backend/, database/, tests/. La estructura definitiva debe adaptarse a la tecnología existente. No cambiar el framework actual sin una razón técnica importante.

```

BALDIRIS/
|
+- README.md
+- AGENTS.md
+- .env.example
+- .gitignore
|
+- docs/
|   +- arquitectura.md
|   +- base-datos.md
|   +- seguridad.md
|   +- roadmap.md
|
+- frontend/
+- backend/

```

```
+ - database/
+ - tests/
```

27. AGENTS.MD

Debe existir un archivo AGENTS.md.

Debe indicar qué es BALDIRIS, arquitectura, tecnologías, reglas de código, reglas de seguridad, reglas de base de datos, reglas multi-tenant, qué archivos puede modificar, qué archivos no debe modificar sin autorización, cómo ejecutar pruebas y cómo ejecutar el proyecto.

28. DESARROLLO POR FASES

No intentar construir absolutamente todo simultáneamente.

FASE 1 - Núcleo BALDIRIS: Login, Usuarios, Roles, Dashboard, Configuración, Navegación. FASE 2 - Administración: Clientes, Proyectos, Ingresos, Gastos, Reservas, Utilidades, Colaboradores, Pagos, Reportes. FASE 3 - Sistema de negocios: Crear negocio, Editar negocio, Activar/desactivar, Usuarios por negocio, Configuración por negocio, Separación de datos. FASE 4 - BALDIRIS POS: Integrar sistema existente K.E. Chicharrón. FASE 5 - K.E. Chicharrón: configurar como primer negocio real. FASE 6 - Producción: Dominio, HTTPS, Servidor, Base de datos, variables de entorno, backups, logs, monitoreo y recuperación ante errores.

Fase 1 - Núcleo BALDIRIS

- Login, Usuarios, Roles, Dashboard, Configuración y Navegación.

Fase 2 - Administración

- Clientes, Proyectos, Ingresos, Gastos, Reservas, Utilidades, Colaboradores, Pagos y Reportes.

Fase 3 - Sistema de negocios

- Crear, editar, activar/desactivar negocios, usuarios por negocio, configuración y separación de datos.

Fase 4 - BALDIRIS POS

- Integrar el sistema existente de K.E. Chicharrón.

Fase 5 - K.E. Chicharrón

- Configurar como primer negocio real dentro de BALDIRIS.

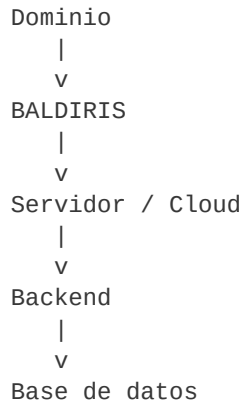
Fase 6 - Producción

- Dominio, HTTPS, servidor, base de datos, variables de entorno, backups, logs, monitoreo y recuperación ante errores.

29. INFRAESTRUCTURA

BALDIRIS debe poder funcionar en una infraestructura profesional.

Arquitectura inicial: Dominio -> BALDIRIS -> Servidor / Cloud -> Backend -> Base de datos. La infraestructura inicial podrá utilizar servicios como Render u otro proveedor cloud adecuado. No contratar infraestructura costosa hasta conocer las necesidades reales del sistema. Primero desarrollar y probar; posteriormente seleccionar el plan adecuado para producción.



30. DOMINIO

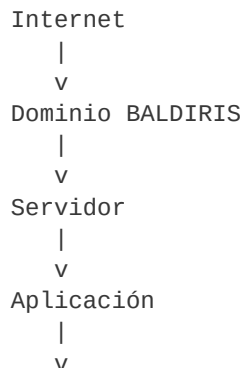
La empresa deberá tener un dominio propio.

Objetivo: baldiris.com o, si no está disponible, baldiris.co u otra alternativa profesional. Antes de comprar: comprobar disponibilidad real, comprobar antecedentes de marca y comprobar que el dominio sea adecuado para una empresa tecnológica. El dominio debe utilizarse para la plataforma y posteriormente para el sitio corporativo de BALDIRIS.

31. PRODUCCIÓN

La plataforma deberá estar preparada para Internet.

Flujo: Internet -> Dominio BALDIRIS -> Servidor -> Aplicación -> Base de datos. Debe utilizar HTTPS. Las credenciales de producción nunca deben almacenarse directamente en el repositorio.



32. GITHUB

GitHub será utilizado como sistema de control de versiones.

Debe existir un repositorio BALDIRIS. Se recomienda trabajar mediante main, develop y feature/* o una estrategia equivalente. Los commits deben ser claros, por ejemplo: feat: crear módulo de clientes; feat: agregar gestión de negocios; fix: corregir autenticación; refactor: separar módulo financiero.

33. BACKUPS

La información empresarial es crítica.

La plataforma deberá contemplar copias de seguridad de la base de datos, recuperación de información, protección contra eliminación accidental y procedimientos de restauración. No depender únicamente de almacenamiento temporal del servidor.

34. ESCALABILIDAD

BALDIRIS debe poder pasar de 1 negocio a 10 y posteriormente a 100+ sin reconstruir completamente el sistema.

La arquitectura debe evitar código duplicado, bases de código independientes innecesarias, configuraciones manuales repetitivas y datos mezclados.

1 negocio -> 10 negocios -> 100+ negocios

35. FUTUROS PRODUCTOS

Después de BALDIRIS POS podrán crearse:

BALDIRIS POS; BALDIRIS Business; BALDIRIS Hotel; BALDIRIS Finance; BALDIRIS AI. Estos productos deberán compartir el núcleo de BALDIRIS cuando sea técnicamente conveniente.

```
BALDIRIS
|
+- BALDIRIS POS
+- BALDIRIS Business
+- BALDIRIS Hotel
+- BALDIRIS Finance
+- BALDIRIS AI
```

36. MODELO COMERCIAL

BALDIRIS podrá trabajar bajo diferentes modelos:

- Implementación: pago único por configurar el sistema.
- Suscripción: pago mensual por utilizar el servicio.
- Licencia: pago por derecho de uso bajo condiciones específicas.
- Desarrollo personalizado: precio independiente para módulos especiales.
- Soporte y mantenimiento: servicio recurrente.

37. PROPIEDAD DEL SOFTWARE

La arquitectura y código reutilizable desarrollado por BALDIRIS deben mantenerse como propiedad de BALDIRIS salvo que exista un acuerdo específico de transferencia.

Los clientes normalmente adquirirán uso del sistema, implementación, configuración, soporte y mantenimiento. No se debe transferir automáticamente el código fuente ni la propiedad intelectual.

38. REGLA PARA NUEVOS CLIENTES

Cuando llegue un nuevo restaurante, no hacer un proyecto separado por restaurante.

En lugar de proyecto-restaurante-1, proyecto-restaurante-2, proyecto-restaurante-3, crear un nuevo negocio dentro de BALDIRIS y configurar el negocio dentro de la plataforma.

```

BALDIRIS
|
v
Negocios
|
v
Nuevo negocio
|
v
Configurar dentro de la plataforma

```

39. EJEMPLO DE CRECIMIENTO

La plataforma debe poder crecer progresivamente.

Inicialmente: BALDIRIS -> K.E. Chicharrón. Después: BALDIRIS -> K.E. Chicharrón, Restaurante 2, Restaurante 3. Después: BALDIRIS -> Restaurantes (K.E. Chicharrón, Restaurante 2, Restaurante 3), Hoteles (Hotel 1), Empresas (Empresa 1, Empresa 2).

```

BALDIRIS
|
+- Restaurantes

```

```

|   +- K.E. Chicharrón
|   +- Restaurante 2
|   +- Restaurante 3
|
+- Hoteles
|   +- Hotel 1
|
+- Empresas
    +- Empresa 1
    +- Empresa 2

```

40. REGLAS PARA CODEX

Codex debe cumplir estas reglas:

- No modificar funcionalidades existentes sin analizarlas primero.
- No borrar archivos importantes sin comprobar su función.
- No introducir dependencias innecesarias.
- No almacenar secretos en el código.
- No crear datos de prueba dentro de producción.
- Todo módulo nuevo debe integrarse con la arquitectura existente.
- Mantener separación entre BALDIRIS y los negocios.
- Respetar el concepto multi-tenant.
- Probar cada funcionalidad nueva.
- Si existe un error, solucionarlo antes de continuar con funcionalidades dependientes.
- No cambiar de tecnología principal sin justificarlo.
- Antes de realizar modificaciones grandes, explicar brevemente qué se va a modificar.

41. METODOLOGÍA DE TRABAJO

Cada nueva funcionalidad deberá seguir:

1. Analizar -> 2. Diseñar -> 3. Implementar -> 4. Probar -> 5. Corregir -> 6. Documentar -> 7.

Integrar. No saltar directamente al código cuando sea necesario analizar primero la arquitectura.

42. CALIDAD

El sistema debe priorizar:

- Seguridad.
- Estabilidad.
- Correcta separación de datos.
- Facilidad de mantenimiento.
- Escalabilidad.
- Experiencia de usuario.
- Velocidad.

- Diseño.

No sacrificar seguridad o integridad de datos únicamente por desarrollar más rápido.

43. ESTADO INICIAL

BALDIRIS comenzará utilizando como punto de partida el sistema existente desarrollado para K.E. Chicharrón.

Ese sistema ya contiene funcionalidades relacionadas con menú, productos, pedidos, carrito, métodos de pago, cocina, estados de pedidos y reportes. Debe analizarse el código existente antes de decidir qué partes serán reutilizadas, refactorizadas o reemplazadas.

44. PRIMER OBJETIVO TÉCNICO

El primer objetivo no es terminar toda la empresa en un solo día.

El primer objetivo es conseguir: BALDIRIS -> Login -> Dashboard -> Administración -> Negocios -> K.E. Chicharrón -> BALDIRIS POS. Cuando esto funcione correctamente, podremos comenzar a incorporar nuevos negocios.

```

BALDIRIS
|
Login
|
Dashboard
|
Administración
|
Negocios
|
K.E. Chicharrón
|
BALDIRIS POS

```

45. PRIMER OBJETIVO COMERCIAL

El primer caso real de BALDIRIS será K.E. Chicharrón.

La meta será poder decir: “K.E. Chicharrón utiliza una solución tecnológica desarrollada y administrada por BALDIRIS.” Posteriormente: “BALDIRIS proporciona soluciones tecnológicas a múltiples negocios.”

46. PRINCIPIO DE CRECIMIENTO

BALDIRIS no debe construirse pensando únicamente en K.E. Chicharrón.

Debe construirse pensando: “K.E. Chicharrón es el primero, no el último.” Cada decisión técnica debe considerar cómo funcionaría si mañana existieran 10, 50 o 100 negocios.

47. SITIO CORPORATIVO FUTURO

Además de la plataforma administrativa, BALDIRIS podrá tener posteriormente un sitio corporativo.

Secciones sugeridas: Inicio, Servicios, Productos, Soluciones, Casos de éxito, Nosotros, Contacto, Iniciar sesión. Desde allí los clientes podrán conocer BALDIRIS y acceder a sus productos.

Inicio | Servicios | Productos | Soluciones | Casos de éxito | Nosotros | Contacto | Iniciar sesión

48. SEPARACIÓN ENTRE SITIO WEB Y PLATAFORMA

Debe distinguirse el sitio público de la plataforma privada.

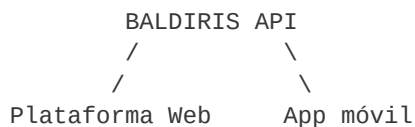
Sitio público: BALDIRIS, página comercial y corporativa. Plataforma privada: app.baldiris... donde administradores y clientes trabajan. No mezclar innecesariamente ambas partes.

Sitio público: BALDIRIS
Plataforma privada: app.baldiris...

49. FUTURA APLICACIÓN MÓVIL

La arquitectura debe dejar abierta la posibilidad de crear posteriormente aplicaciones Android/iOS.

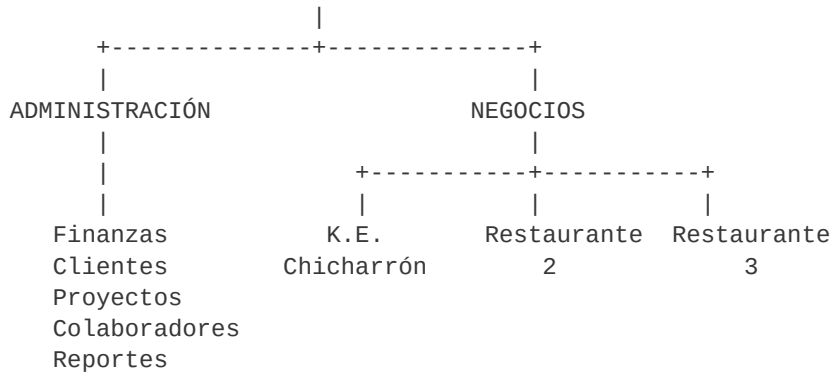
La aplicación móvil podrá consumir la misma API del sistema. Conceptualmente: BALDIRIS API -> Plataforma Web y App móvil.



50. RESULTADO FINAL ESPERADO

El objetivo final es obtener una plataforma donde BALDIRIS tenga una capa de administración central y una capa de negocios con múltiples empresas.

La estructura final debe permitir Administración con Finanzas, Clientes, Proyectos, Colaboradores y Reportes, y Negocios con K.E. Chicharrón, Restaurante 2, Restaurante 3 y futuras empresas.



51. ORDEN DE IMPLEMENTACIÓN OBLIGATORIO

Codex debe seguir aproximadamente este orden:

PASO 1: Analizar proyecto existente. PASO 2: Identificar tecnologías utilizadas. PASO 3: Identificar frontend. PASO 4: Identificar backend. PASO 5: Identificar base de datos. PASO 6: Identificar autenticación. PASO 7: Documentar arquitectura actual. PASO 8: Crear núcleo BALDIRIS. PASO 9: Crear autenticación. PASO 10: Crear Dashboard. PASO 11: Crear Administración. PASO 12: Crear módulo Negocios. PASO 13: Implementar arquitectura multi-tenant. PASO 14: Integrar K.E. Chicharrón. PASO 15: Probar separación de datos. PASO 16: Preparar producción. PASO 17: Configurar dominio. PASO 18: Configurar servidor. PASO 19: Configurar base de datos de producción. PASO 20: Realizar pruebas finales.

Secuencia completa

1. Analizar proyecto existente
2. Identificar tecnologías utilizadas
3. Identificar frontend
4. Identificar backend
5. Identificar base de datos
6. Identificar autenticación
7. Documentar arquitectura actual
8. Crear núcleo BALDIRIS
9. Crear autenticación
10. Crear Dashboard
11. Crear Administración
12. Crear módulo Negocios
13. Implementar arquitectura multi-tenant
14. Integrar K.E. Chicharrón
15. Probar separación de datos
16. Preparar producción
17. Configurar dominio
18. Configurar servidor

19. Configurar base de datos de producción

20. Realizar pruebas finales

52. INSTRUCCIÓN FINAL PARA CODEX

Antes de comenzar a desarrollar, NO empieces creando archivos aleatoriamente.

Primero: inspecciona todo el proyecto disponible; identifica la tecnología utilizada; identifica la estructura actual; identifica la base de datos; identifica las funcionalidades existentes; identifica qué puede reutilizarse; identifica qué debe refactorizarse; compara la arquitectura actual con este documento; propone el primer conjunto de cambios; después comienza la implementación. El proyecto debe evolucionar progresivamente hasta convertirse en BALDIRIS, una plataforma tecnológica empresarial multi-negocio, escalable y preparada para convertirse en la infraestructura tecnológica de múltiples empresas.

53. FRASE CENTRAL DEL PROYECTO

“BALDIRIS no será solamente un software. BALDIRIS será la empresa tecnológica que desarrolla, administra y proporciona soluciones digitales para negocios.”

CIERRE

Este manual constituye el plano inicial del proyecto BALDIRIS. Debe mantenerse actualizado cuando se tomen decisiones arquitectónicas importantes. Las ampliaciones futuras deben respetar la visión central: BALDIRIS como empresa tecnológica y plataforma escalable, con múltiples negocios aislados y administrados desde un núcleo empresarial común.